CI Digital / INATEL

Sistema-em-Chip para Internet-das-coisas: desenvolvimento com base em PicoRV32, AXI-Lite, verificação em UVM e validação em FPGA

T1 / 2025 - Grupo 1 - Cícero

Este trabalho orientado 2 faz parte das exigências para aprovação na disciplina SD292, do módulo avançado (módulo 2), referente ao treinamento CI Digital, realizado no INATEL.

Santa Rita do Sapucaí – MG

Janeiro / 2026

PROGRAMA

O Programa CI Digital é uma iniciativa do Ministério de Ciência, Tecnologia e Inovação (MCTI) junto à Softex para capacitação de profissionais para a indústria brasileira de semicondutores.

O programa destina-se à capacitação de até 350 profissionais para a área de microeletrônica com intuito de posicionar o país como um centro competente de desenvolvimento de projetos de circuitos integrados digitais de alta complexidade e valor agregado.

Os principais objetivos do CI Digital são:

Realizar capacitação tecnológica em âmbito nacional visando à formação prática e teórica em linguagens de descrição de hardware, simulação e ferramentas de projeto para a área de microeletrônica;

Formar projetistas aptos a atuarem no mercado de projetos de front-end para circuitos integrados digitais, com o desenvolvimento e a verificação de arquiteturas digitais avançadas.

ORIENTADORES

Elivander Judas Tadeu Pereira	elivander.pereira@inatel.br
Letícia Carneiro de Souza	leticia.c@inatel.br
Felipe Gustavo de Freitas Rocha	felipef.rocha@inatel.br

COLABORADORES

Ana Carolina Vivian dos Reis	carolinavivian2009@gmail.com
Bernard Chemitoua Mibankanina	bernardchemi63@gmail.com
Cícero Luiz Alves Cunha	eng.cicerocunha@gmail.com
Daniel Quiteque	danielq@get.inatel.br
Felipe Ribeiro Lobato	felipe.ribeiro@gec.inatel.br
Felippe Silvério de Castro	felippesilverio.castro@gmail.com
Gabriel Caynã Diniz Laraia	gabriel.laraia@gmail.com
Gabriele Fidélis Jorge	gabfidelis@gmail.com
Guilherme Pires Piedade	guilherme.pirespiedadep@gmail.com
Igor Azevedo Cintra	engenheiroigorcintra@outlook.com
Letícia da Silva Souza	eng.leticia@outlook.com
Lucas da Silveira Ross	lucassilveiraross@gmail.com
Luiz Renault Leite Rodrigues	luiz.rodrigues@dtel.inatel.br
Luiz Ricardo Pires	luizrpires@gmail.com
Marcelo Pelagio Pontes Morais	curso.cidigital2025@gmail.com
Maysa Freire dos Santos	maysafss@gmail.com
Natanael Irland de Souza	natanaelirland@gmail.com
Paulo Eduardo Pereira Maia	paulo.eduardomaia@hotmail.com
Rafael da Silva Macêdo	rafael.smjf@hotmail.com
Sinal Exaucé Macoungou Nzaou	sinal.exauce@get.inatel.br

SUMÁRIO

	APRESENTAÇÃO	**4**
1	**INTRODUÇÃO**	**5**
2	**PROBLEMATIZAÇÃO**	**5**
3	**FUNDAMENTAÇÃO TEÓRICA**	**5**
3.1	Sistema-em-chip no contexto de internet das coisas	5
3.2	Arquitetura RISC-V e núcleo picoRV32	6
3.3	Validação de sistemas digitais com FPGA	6
3.4	Verificação funcional com metodologia de verificação universal	7
4	**JUSTIFICATIVA**	**8**
5	**OBJETIVOS**	**8**
5.1	Objetivo geral	8
5.2	Objetivos específicos	8
6	**METODOLOGIA**	**9**
6.1	Recursos disponíveis	9
6.2	Equipes de trabalho e competências	9
6.3	Cronograma de projeto	11
6.4	Arquitetura do SoC proposto	12
6.5	Funcionamento global do sistema	13
6.6	Núcleo PicoRV32	14
6.7	Barramento AXI-Lite	15
6.8	Memórias on-chip	16
6.9	Bootloader	17
6.10	Subsistema de periféricos	21
6.11	Mapa de memória	21
6.12	Subsistema de periféricos	22
6.13	Controlador de interrupção	24
6.14	Verificação funcional do núcleo RISC-V	25
6.15	Estrutura do ambiente UVM e sequencer	28
6.16	Driver, monitor e scoreboard do ambiente UVM	33
7	**RESULTADOS E DISCUSSÕES**	**38**
7.1	Design	38
7.2	Verificação funcional, implementação em FPGA e UVM	39
8	**CONCLUSÃO**	**42**
	REFERÊNCIAS	**43**

APRESENTAÇÃO

Este trabalho propôs o desenvolvimento de um **Sistema-em-Chip (SoC)** voltado a aplicações de **Internet das Coisas (IoT)**, implementado em FPGA com o kit **Arty Z7-20**, utilizando como núcleo de processamento o processador **PicoRV32**, compatível com a arquitetura **RISC-V de 32 bits**. O SoC é estruturado de forma modular, com interconexão interna baseada no protocolo **AXI-Lite**, permitindo comunicação padronizada entre o processador, memórias e periféricos. O sistema possui um **módulo bootloader**, o qual permite preparar o ambiente, gravar o programa principal e carregar firmwares sem precisar regravar a FPGA.

A escolha por uma arquitetura SoC com barramento AXI-Lite se justifica por sua ampla adoção em sistemas embarcados modernos, simplicidade relativa quando comparado ao AXI completo, e adequação a sistemas com periféricos mapeados em memória e largura de dados de 32 bits. O foco do projeto é a construção de uma plataforma didática, porém realista, capaz de executar firmware bare-metal e interagir com periféricos típicos de aplicações IoT.

O SoC integra os seguintes blocos principais: núcleo de processamento PicoRV32; interconexão AXI-Lite; memórias on-chip (ROM e RAM); periféricos básicos de entrada/saída (GPIO, UART, SPI e I2C); temporizador de sistema (timer); controlador de interrupções; infraestrutura de clock e reset.

Na verificação funcional do SoC, cada requisito funcional está vinculado a pelo menos um teste e a métricas de coverage. Os testes são descritos de forma sistemática e planejados, a fim de validar o comportamento do sistema, em conformidade com a especificação. Sua principal finalidade é garantir que todos os requisitos funcionais do SoC sejam adequadamente exercidos, reduzindo riscos de falhas e assegurando a confiabilidade do projeto antes da validação em FPGA.

Os colaboradores

1 INTRODUÇÃO

O avanço da Internet das Coisas (Internet of Things – IoT) impulsiona o desenvolvimento de sistemas embarcados cada vez mais integrados, eficientes e confiáveis. Neste contexto, Sistemas-em-Chip (System-on-Chip – SoC) tornaram-se a principal abordagem para integração de processador, memória e periféricos em uma única plataforma.

2 PROBLEMATIZAÇÃO

O desenvolvimento de SoCs envolve desafios relacionados à integração de hardware e software, além da necessidade de metodologias robustas de verificação funcional. Projetos acadêmicos frequentemente negligenciam a etapa de verificação estruturada, comprometendo a confiabilidade do sistema.

3 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica do Sistema-em-Chip (SoC) envolve a integração de processador, memória, periféricos e barramentos de comunicação em uma única arquitetura digital. Essa abordagem permite a construção de sistemas embarcados completos, com maior eficiência, menor latência e melhor aproveitamento de recursos. O uso de arquiteturas abertas, como RISC-V, e de protocolos padronizados de interconexão, como o AXI-Lite, garante modularidade, escalabilidade e facilidade de verificação. Dessa forma, o entendimento dos princípios arquiteturais e das técnicas de descrição e validação em nível RTL é essencial para o desenvolvimento consistente de um SoC.

3.1 Sistema-em-chip no contexto de internet das coisas

A Internet das Coisas (Internet of Things – IoT) refere-se à integração de dispositivos físicos dotados de sensores, atuadores, capacidade de processamento e conectividade, possibilitando a coleta e o processamento distribuído de dados. Nesse contexto, requisitos como baixo consumo de energia, integração funcional, confiabilidade e custo reduzido tornam-se fundamentais no projeto de sistemas embarcados (ATZORI; IERA; MORABITO, 2010).

Os Sistemas-em-Chip (System-on-Chip – SoC) constituem uma abordagem arquitetural que integra, em um único circuito, processador, memórias, periféricos e interfaces de comunicação.

Essa integração reduz a complexidade do sistema, melhora a eficiência energética e diminui latências de comunicação interna (WOLF, 2012). Em aplicações IoT, SoCs favorecem arquiteturas orientadas a eventos, com uso intensivo de interrupções, o que contribui para maior eficiência e responsividade do sistema.

Além disso, o uso de periféricos mapeados em memória, acessados por barramentos padronizados, simplifica o desenvolvimento de software e facilita a aplicação de metodologias formais de verificação, tornando o SoC uma solução adequada para dispositivos IoT modernos (MARWEDEL, 2018).

3.2 Arquitetura RISC-V e núcleo picoRV32

A arquitetura RISC-V é uma arquitetura de conjunto de instruções aberta e modular, projetada para permitir extensibilidade, simplicidade e independência de licenças proprietárias. Sua especificação é pública, o que possibilita a implementação de núcleos customizados para diferentes aplicações, tanto acadêmicas quanto industriais (WATERMAN; ASANOVIĆ, 2019).

O subconjunto RV32I define uma arquitetura de 32 bits com um conjunto básico de instruções, suficiente para aplicações embarcadas e sistemas de controle. A simplicidade desse subconjunto favorece implementações compactas e eficientes, sendo particularmente adequada para sistemas com restrições de área e consumo, como dispositivos IoT (PATTERSON; HENNESSY, 2021).

O núcleo PicoRV32 é uma implementação open-source da arquitetura RISC-V RV32I, caracterizada por sua simplicidade estrutural, baixo consumo de recursos lógicos e facilidade de integração em projetos de SoC. O núcleo suporta execução bare-metal e integração com barramentos como o AXI-Lite, sendo amplamente utilizado em projetos educacionais e de prototipação rápida (WOLF; PICORV32, 2023). Essas características justificam sua adoção como unidade central de processamento neste trabalho.

3.3 Validação de sistemas digitais com FPGA

Field-Programmable Gate Arrays (FPGAs) são dispositivos reconfiguráveis que permitem a implementação física de circuitos digitais descritos em linguagens de descrição de hardware. A

validação em FPGA possibilita a execução do sistema em condições reais de operação, permitindo a observação de aspectos como temporização, concorrência e interação com periféricos físicos (MAXFIELD, 2009).

No desenvolvimento de SoCs, a FPGA atua como uma plataforma de prototipação rápida, viabilizando a execução de firmware e a análise da integração entre hardware e software. Essa etapa é essencial para identificar problemas que podem não ser detectados em simulação, como comportamentos associados à inicialização, temporização e comunicação externa (MARWEDEL, 2018).

Dessa forma, a validação em FPGA complementa a verificação funcional realizada em ambiente de simulação, fornecendo evidências experimentais de que o sistema atende aos requisitos especificados. No presente trabalho, o uso do kit FPGA **Arty Z7-20** permitiu a implementação física do SoC e a validação de seu funcionamento por meio de interfaces reais, como UART e GPIO.

3.4 Verificação funcional com metodologia de verificação universal

A verificação funcional é uma etapa crítica no desenvolvimento de sistemas digitais complexos, especialmente em projetos de SoC, nos quais múltiplos componentes interagem de forma concorrente. Métodos tradicionais de simulação com base apenas em estímulos dirigidos tornam-se insuficientes à medida que a complexidade do sistema aumenta (FOSTER; KROPF; LIPPMANN, 2004).

SystemVerilog estende a linguagem Verilog ao incorporar recursos avançados para abstração, orientação a objetos e verificação. Sobre essa linguagem foi desenvolvida a metodologia de verificação universal (Universal Verification Methodology-UVM), que define uma estrutura padronizada para ambientes de verificação reutilizáveis e escaláveis, baseada em transações e métricas de cobertura (ACCELLERA, 2020).

A UVM promove a separação clara entre geração de estímulos, condução de sinais, observação do comportamento do sistema e checagem dos resultados. Componentes como agentes, sequências, monitores, scoreboards e mecanismos de cobertura funcional permitem uma abordagem sistemática e mensurável da verificação. A camada de abstração de registro (Register Abstraction Layer-RAL) possibilita a modelagem dos registradores do sistema em

alto nível, facilitando o acesso, a geração automática de testes e a coleta de cobertura de registradores (ACCELLERA, 2020).

No contexto deste trabalho, a utilização de SystemVerilog e UVM permitiu a verificação estruturada do SoC, abrangendo tanto o protocolo do barramento AXI-Lite quanto o funcionamento integrado dos periféricos, reduzindo riscos e aumentando a confiabilidade do sistema antes da validação em FPGA.

4 JUSTIFICATIVA

A adoção de um SoC com base em RISC-V, aliado à verificação com UVM e validação em FPGA, proporciona alinhamento com práticas industriais e valor acadêmico ao projeto.

5 OBJETIVOS

Ao final deste trabalho, os colaboradores esperam alcançar os objetivos descritos nas seções seguintes.

5.1 Objetivo geral

- Desenvolver um Sistema-em-Chip para Internet-das-coisas com base em PicoRV32, AXI-Lite, verificação em UVM e validação em FPGA.

5.2 Objetivos específicos

- Projetar arquitetura SoC;
- Implementar periféricos;
- Fazer a integração do SoC;
- Realizar verificação funcional do SoC;
- Validar em FPGA.

6 METODOLOGIA

A metodologia adotada neste projeto segue uma abordagem incremental e iterativa, comum em projetos de Sistemas-em-Chip, permitindo o desenvolvimento progressivo do hardware, verificação funcional e validação em FPGA.

6.1 Recursos disponíveis

No desenvolvimento do projeto utilizou-se recursos de hardware e software adequados às etapas de projeto, verificação e validação do Sistema-em-Chip, conforme mostra a Tabela 1. A implementação física foi realizada na plataforma de desenvolvimento **Arty Z7-20** da Digilent, projetada em torno do SoC Zynq-7000 da AMD (Xilinx) e processador dual-core ARM Cortex-A9, com a flexibilidade da lógica programável de um FPGA.

O desenvolvimento do hardware digital foi conduzido em **Verilog**, enquanto na verificação funcional utilizou-se **SystemVerilog e UVM**, executadas em simulador compatível (**Vivado**). A síntese e implementação em FPGA foram realizadas com as ferramentas **Quartus** e **Vivado**.

O desenvolvimento do firmware efetuar-se-á por meio de uma **toolchain RISC-V**, possibilitando a execução de software bare-metal no núcleo PicoRV32 e a validação da integração hardware–software.

Tabela 1 – Recursos disponíveis para o desenvolvimento do projeto

RECURSO	DESCRIÇÃO	FINALIDADE
kit FPGA Arty Z7-20	Placa FPGA dual-core ARM Cortex-A9	implementação e validação do SoC
Quartus	Ferramenta de síntese	Síntese e análise temporal
Vivado	Simulador SystemVerilog	Verificação funcional UVM
Toolchain RISC-V	Compilador C	Desenvolvimento do firmware

Fonte: Autoria própria.

6.2 Equipes de trabalho e competências

A Tabela 2 mostra as equipes de design, verificação e documentação, divididas em subgrupos (A, B, C, D, E, F e G), formadas de acordo com as competências e habilidades dos colaboradores.

Tabela 2 – Equipes de trabalho e subgrupos

	Líder	COLABORADORES	EQUIPES						
			DESIGN		DOCUMENTAÇÃO		VERIFICAÇÃO		
			A	B	C	D	E	F	G
01		Ana Carolina Vivian dos Reis							X
02		Bernard Chemitoua Mibankanina							X
03	D	Cícero Luiz Alves Cunha				X			
04		Daniel Quiteque							
05		Felipe Ribeiro Lobato						X	
06		Felippe Silvério de Castro	X						
07		Gabriel Caynã Diniz Laraia				X			
08		Gabriele Fidélis Jorge			X				
09		Guilherme Pires Piedade					X		
10	G	Igor Azevedo Cintra							X
11		Letícia da Silva Souza						X	
12	A	Lucas da Silveira Ross	X						
13	E	Luiz Renault Leite Rodrigues					X		
14		Luiz Ricardo Pires		X					
15		Marcelo Pelagio Pontes Morais							X
16	C	Maysa Freire dos Santos			X				
17		Natanael Irland de Souza							X
18	F	Paulo Eduardo Pereira Maia						X	
19	B	Rafael da Silva Macêdo		X					
20		Sinal Exaucé Macoungou Nzaou							X

Legenda: A - Código funcional núcleo e periférico; B - Código funcional integração; C - Documentação funcional; D - Documentação verificação; E - Verificação funcional; F - Verificação UVM - estrutura e sequencer; G - Verificação UVM - monitor, driver e scoreboard.
Fonte: Autoria própria.

A equipe de design é responsável pela definição da arquitetura do SoC e pela implementação do hardware digital, abrangendo a descrição RTL do processador, do barramento interno, das memórias e dos periféricos. Essa equipe deve possuir competências em arquitetura de sistemas digitais, linguagens de descrição de hardware e integração de módulos, assegurando que o projeto atenda aos requisitos funcionais e às restrições de desempenho e recursos.

A equipe de verificação tem como principal responsabilidade assegurar que o sistema implementado esteja em conformidade com a especificação. Por meio da construção de ambientes de verificação estruturados, baseados em SystemVerilog e UVM, essa equipe define estratégias de teste, desenvolve sequências, monitores, scoreboards e mecanismos de cobertura, garantindo a detecção sistemática de falhas. A verificação desempenha papel central na redução de riscos, especialmente em projetos de SoC, nos quais erros de integração podem ser complexos e custosos de corrigir.

A equipe de documentação é responsável por registrar de forma clara e consistente todas as etapas do projeto, desde a concepção arquitetural até os resultados de validação. Sua atuação

assegura a rastreabilidade entre requisitos, decisões de projeto, testes e resultados, além de garantir conformidade com normas da Associação Brasileira de Normas Técnicas (ABNT), e facilitar a comunicação técnica (UFMG, 2025). Uma documentação bem estruturada contribui diretamente para a compreensão, manutenção e reprodutibilidade do sistema desenvolvido.

Mesmo em projetos acadêmicos conduzidos por um único autor, a organização do trabalho segundo essas três frentes reflete práticas consolidadas da indústria, promovendo uma abordagem sistemática e profissional para o desenvolvimento de Sistemas-em-Chip.

O desenvolvimento de um Sistema-em-Chip envolve a integração de múltiplas atividades técnicas, exigindo uma organização clara das responsabilidades e competências associadas a cada etapa do projeto, conforme mostra a Tabela 3. Nesse contexto, a atuação coordenada das equipes de design, verificação e documentação é fundamental para garantir a funcionalidade, confiabilidade e rastreabilidade do sistema desenvolvido.

Tabela 3 – Equipes de trabalho e respectivas responsabilidades e competências

Equipe	Responsabilidades	Competências
Design	Arquitetura SoC e RTL Verilog	Arquitetura digital, Verilog
Verificação	Testbench UVM	SystemVerilog, UVM
Documentação	Redação e revisão	Normas ABNT (UFMG, 2025)

Fonte: Autoria própria.

6.3 Cronograma de projeto

Na Tabela 4 tem-se o cronograma do projeto que foi estruturado para um período de seis semanas, considerando a complexidade técnica do desenvolvimento de um Sistema-em-Chip e a necessidade de integração entre projeto, verificação, síntese e implementação em FPGA .

Nas duas primeiras semanas serão realizados o estudo dos conceitos fundamentais e a definição da arquitetura do sistema. A especificação dos módulos e a implementação da interconexão AXI-Lite ocorrerão na sequência, permitindo o início da integração do SoC.

As etapas de desenvolvimento do firmware e do ambiente de verificação com UVM serão conduzidas de forma paralela à implementação dos periféricos, prática comum em projetos de sistemas embarcados. As semanas finais serão dedicadas à verificação funcional, síntese do projeto, validação experimental em FPGA e documentação do trabalho final.

O cronograma é intensivo e elaborado com base em reutilização de IPs, desenvolvimento incremental e paralelismo entre implementação, verificação e firmware, práticas comuns em projetos de SoC.

Tabela 4 - Cronograma das etapas e atividades do projeto

Etapas	Descrição	Semanas					
		S1	S2	S3	S4	S5	S6
1	Estudo e planejamento inicial	X	X				
2	Definição da arquitetura do SoC		X	X			
3	Especificação dos módulos e registradores			X			
4	Implementação do interconector AXI-Lite			X	X		
5	Implementação de memórias on-chip			X			
6	Implementação dos periféricos				X	X	
7	Integração do SoC				X		
8	Desenvolvimento do firmware				X	X	
9	Ambiente de verificação (UVM)			X	X	X	X
10	Verificação funcional e coverage				X	X	X
11	Síntese e implementação em FPGA				X	X	X
12	Validação experimental em FPGA					X	X
13	Análise de resultados e documentação	X	X	X	X	X	X

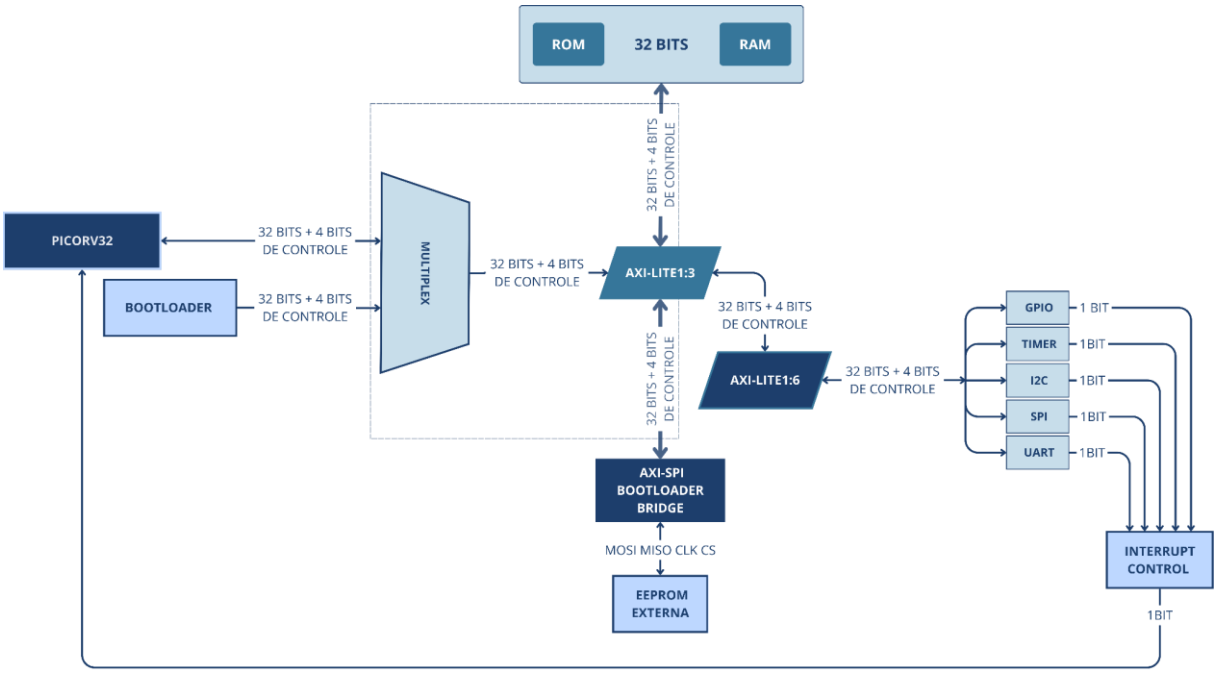
Fonte: Autoria própria.

6.4 Arquitetura do SoC proposto

A Figura 1 apresenta a arquitetura do Sistema-em-Chip desenvolvido neste trabalho, com base no processador PicoRV32, com interconexão interna por meio do barramento AXI-Lite. O sistema integra memórias on-chip, periféricos mapeados em memória e um controlador de interrupções, permitindo comunicação orientada a eventos. As larguras de endereço e dados do barramento são de 32 bits, compatíveis com a arquitetura RISC-V RV32. O sistema opera com clock de 100 MHz, proveniente do kit FPGA Arty Z7-20, e reset síncrono, garantindo previsibilidade temporal e simplicidade de implementação (LARAIA et al., 2026).

A arquitetura funcional do Sistema-em-Chip fundamenta-se em organização modular hierárquica, na qual cada bloco exerce função específica dentro do conjunto do sistema. Essa organização distingue os domínios de processamento, interconexão, armazenamento, interface externa e gerenciamento de eventos, promovendo separação clara de responsabilidades e previsibilidade operacional.

Figura 1 – Arquitetura em blocos do Sistema-em-Chip proposto



Fonte: Autoria própria.

A estrutura do SoC é composta por: núcleo de processamento (PicoRV32); infraestrutura de interconexão AXI-Lite; subsistema de memórias on-chip; subsistema de periféricos (GPIO, UART, SPI, I2C e Timer); módulo bootloader; e controlador de interrupções.

O núcleo PicoRV32 constitui a unidade central de processamento, responsável pela execução do firmware e pela coordenação das operações do sistema. A comunicação entre os blocos é realizada por meio da infraestrutura de interconexão AXI-Lite, que estabelece modelo uniforme de acesso entre processamento, armazenamento e interface externa.

Cada módulo ocupa faixa específica do espaço de endereçamento e interage com os demais exclusivamente por meio da infraestrutura de interconexão, assegurando desacoplamento estrutural entre processamento, armazenamento e interface externa.

6.5 Funcionamento global do sistema

O funcionamento global do SoC descreve o comportamento dinâmico do sistema a partir de sua organização arquitetural. Em aplicações IoT, esse modelo possibilita a atuação do SoC como dispositivo embarcado responsável pela coleta, processamento local e transmissão de dados. Os

sensores e atuadores conectados aos periféricos integram-se ao fluxo operacional por meio da infraestrutura de interconexão (LARAIA et al., 2026).

A operação inicia-se com a aplicação do sinal de reset, que direciona o contador de programa ao vetor de inicialização localizado na memória on-chip. O firmware executa rotinas de configuração inicial, incluindo a definição do ponteiro de pilha e a preparação dos periféricos essenciais.

Concluída essa etapa, o núcleo PicoRV32 passa a executar o programa principal. Todas as interações com memórias e periféricos ocorrem por meio de transações mapeadas em memória.

O sistema opera segundo ciclo contínuo de aquisição, processamento e resposta. Dados provenientes do ambiente externo são disponibilizados ao núcleo por registradores mapeados em memória, permitindo que o firmware processe informações e atualize saídas digitais ou realize comunicação pelas interfaces disponíveis.

Eventos internos ou externos podem gerar requisições de interrupção. O controlador de interrupções centraliza essas requisições e sinaliza o núcleo para execução de rotinas específicas, promovendo resposta eficiente e previsível, sem necessidade de monitoramento contínuo.

6.6 Núcleo PicoRV32

O núcleo PicoRV32 implementa a arquitetura RISC-V RV32I e constitui a unidade central de processamento do SoC. Sua função é executar o firmware, controlar o fluxo de instruções e coordenar o acesso às memórias e aos módulos mapeados no espaço de endereçamento do sistema.

A Tabela 5 apresenta os principais módulos RTL que compõem o núcleo na arquitetura implementada (LARAIA et al., 2026). No desenvolvimento RTL, o núcleo foi integrado preservando sua estrutura modular original, mantendo separação entre núcleo de execução, banco de registradores e extensões aritméticas opcionais. A adaptação ao protocolo AXI-Lite foi realizada por meio de wrapper e adaptador dedicados, responsáveis por converter a interface nativa de memória do núcleo para os canais do barramento. A metodologia adotada consistiu na manutenção do core como bloco independente e reutilizável, evitando modificações internas

e isolando a lógica computacional da infraestrutura de interconexão. Essa estratégia assegura um desequilíbrio estrutural entre processamento e transporte de dados.

Tabela 5 - Arquivos RTL e módulos do subsistema do núcleo PicoRV32

Arquivo (.v)	Módulo Principal	Descrição Funcional
picorv32.v	picorv32	Core RISC-V RV32I principal (decode/execute e acesso à memória)
picorv32.v	picorv32_regs	Banco de registradores do PicoRV32 (x0..x31)
picorv32.v	picorv32_pcpi_mul	Coprocessador PCPI de multiplicação iterativa
picorv32.v	picorv32_pcpi_fast_mul	Coprocessador PCPI de multiplicação com caminho rápido
picorv32.v	picorv32_pcpi_div	Coprocessador PCPI para divisão/módulo
picorv32.v	picorv32_axi	Wrapper do core com interface AXI mestre
picorv32.v	picorv32_axi_adapter	Adaptador entre interface nativa de memória e canal AXI

Fonte: Autoria própria.

6.7 Barramento AXI-Lite

O barramento AXI-Lite constitui a infraestrutura de interconexão interna do SoC, responsável pela comunicação entre o núcleo de processamento e os módulos escravos mapeados em memória (ARM, 2010; ARM, 2011). Na arquitetura implementada, o PicoRV32 atua como mestre único, enquanto memórias e periféricos operam como escravos selecionados por lógica determinística de decodificação de endereços. Essa configuração simplifica a interconexão e favorece a previsibilidade temporal das transações.

A implementação RTL foi estruturada de forma hierárquica, concentrando no interconector as funções de decodificação, roteamento e correspondência transacional das respostas. A metodologia adotada baseou-se na centralização da lógica de interconexão em módulos específicos, evitando a replicação de lógica de roteamento nos blocos funcionais.

6.7.1 Características estruturais

O barramento opera com largura de dados e endereços de 32 bits, compatível com a arquitetura RV32I (WATERMAN; ASANOVIĆ, 2019). As transações são realizadas em palavras alinhadas.

O protocolo organiza operações em canais independentes de leitura e escrita, estruturados pelo mecanismo VALID/READY (ARM, 2010). O suporte a escrita parcial é provido pelo sinal WSTRB, permitindo atualização seletiva de bytes. Os sinais RRESP e BRESP retornam o status das operações.

6.7.2 Arquitetura do interconector

A Tabela 6 apresenta os arquivos RTL que compõem a infraestrutura de interconexão (LARAIA et al., 2026). O interconector realiza o roteamento das transações por meio de comparação de faixas de endereço previamente definidas no mapa de memória. Sua implementação foi segmentada em dois níveis, separando subsistema de memória e subsistema de periféricos.

Tabela 6 - Arquivos RTL e módulos do subsistema do barramento AXI-Lite

Arquivo (.v)	Módulo Principal	Descrição Funcional
axi_1_2.v	axi_lite_ic_1x2	Interconector AXI-Lite 1×2 para separação entre memória e periféricos.
axi_interconect_1_6.v	axi_lite_interconnect_1x6	Interconector AXI-Lite 1×6 para decodificação e roteamento aos periféricos.
top_teste.v	soc_top_teste_mem_periph	Top-level do SoC: integra CPU, interconnects, RAM, periféricos e IRQs.

Fonte: Autoria própria.

A correspondência entre requisição e resposta é mantida dinamicamente, garantindo consistência transacional. O controle de fluxo é regulado pela propagação dos sinais READY conforme a latência de cada módulo.

6.8 Memórias on-chip

As memórias on-chip constituem o subsistema de armazenamento do SoC e sustentam tanto o processo de inicialização quanto a execução do firmware.

A arquitetura contempla região destinada ao vetor de reset e rotinas iniciais, bem como região utilizada como área de execução, pilha e armazenamento de dados. O acesso ocorre exclusivamente por transações mapeadas em memória mediadas pela infraestrutura AXI-Lite.

A Tabela 7 apresenta os arquivos RTL associados ao subsistema de memória (LARAIA et al., 2026). No desenvolvimento RTL, a memória RAM foi implementada como módulo escravo AXI-Lite independente, contendo lógica interna de armazenamento desacoplada da interface de barramento. O suporte a escrita parcial foi integrado por meio do sinal WSTRB, mantendo compatibilidade com o protocolo.

A metodologia adotada consistiu na implementação da memória como bloco modular reutilizável, preservando a separação entre lógica de armazenamento e infraestrutura de interconexão.

Tabela 7 - Arquivos RTL e módulos do subsistema de memória

Arquivo (.v)	Módulo Principal	Descrição Funcional
axi_mem.v	axi_lite_ram	Memória RAM mapeada em AXI-Lite com suporte a escrita parcial via WSTRB.

Fonte: Autoria própria.

6.9 Bootloader

Na realização do SoC foi projetada uma arquitetura que inicialmente não previa o bootloader. Dessa forma, foram criados dois arquivos AXI-Lite contendo as conexões do barramento AXI além do top level que interligava os componentes do SoC. Nesse modelo, a fim de facilitar o desenvolvimento e a configuração do SoC, foi criado um arquivo de cabeçalho contendo os endereços e máscaras de endereçamento dos componentes, de forma que qualquer alteração em um periférico exigisse apenas a atualização desse arquivo central (LARAIA et al., 2026).

6.9.1 Descritivo do circuito de bootloader

O módulo axi_bootloader_memcpy implementa, em hardware, um mecanismo de boot por cópia de memória (“memcpy”) utilizando um master AXI4-Lite. A função principal é copiar um bloco de dados armazenado em uma região de leitura (tipicamente uma EEPROM/Flash acessada via uma janela AXI-SPI) para a RAM do sistema, viabilizando a inicialização do firmware sem depender de software prévio em execução (LARAIA et al., 2026).

O acionamento é feito pelo sinal boot_pin:

- boot_pin = 1: o bootloader entra em atividade e executa a cópia de WORDS palavras de 32 bits;

- boot_pin = 0: o bootloader permanece inativo (idle). Se estava em **finish**, retorna a IDLE e limpa flags de status, conforme a lógica implementada.

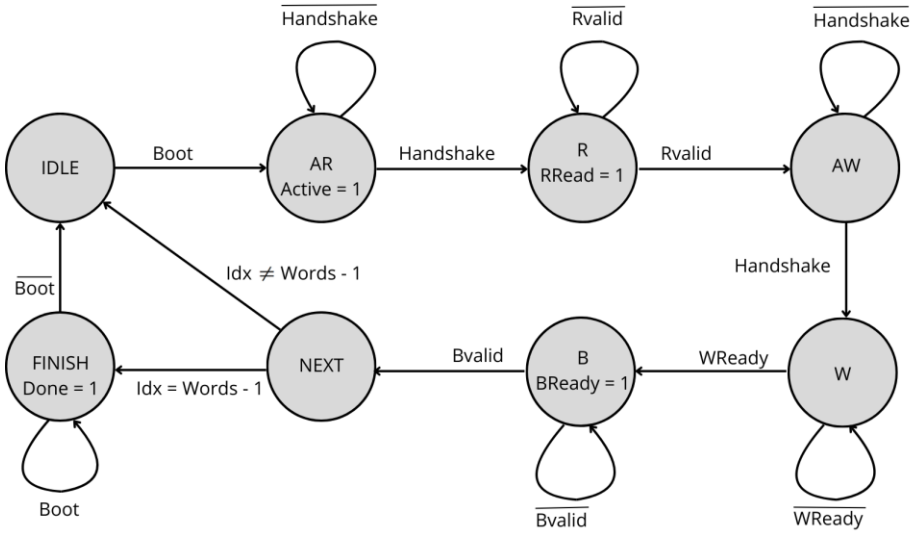
O processo de cópia é sequencial e síncrono, palavra por palavra:

- Lê uma palavra 32-bit da origem (SRC);
- Escreve essa palavra na RAM (DEST);
- Incrementa índice e repete até completar WORDS.

A máquina de estado finito (Finite State Machine - FSM) mostrada na Figura 2 possui 8 estados: ST_IDLE; ST_AR; ST_R; ST_AW; ST_W; ST_B; ST_NEXT; ST_FINISH. E, logo em seguida, é realizada uma descrição funcional detalhada, do bootloader, pela ordem de execução.

A máquina de estados foi estruturada de forma sequencial e não pipelineada, garantindo que apenas uma transação de leitura e uma de escrita estejam ativas por vez. Essa decisão simplifica a implementação, evita dependência de buffers intermediários complexos e assegura conformidade clara com o protocolo AXI-Lite.

Figura 2 – FSM desenvolvida para o bootloader



Fonte: Autoria própria.

O módulo axi_bootloader_memcpy implementa o mecanismo de boot por hardware cuja função é copiar um bloco de palavras de 32 bits a partir de uma região de memória mapeada

(tipicamente uma EEPROM acessada via ponte AXI-SPI) para a RAM do sistema. O bloco atua como master AXI-Lite e executa uma sequência determinística de transações de leitura e escrita, controladas por uma máquina de estados finita (FSM). A operação é iniciada quando boot_pin é ativado e permanece ativa até que o número configurado de palavras (WORDS) seja transferido.

O estado inicial (ST_IDLE) representa a condição de repouso do sistema. Nele, todos os sinais de controle do barramento são mantidos inativos, evitando qualquer transação espúria. A existência desse estado é fundamental para garantir que o módulo não interfira no barramento enquanto boot_pin estiver desabilitado. Quando boot_pin é acionado, a FSM inicializa o índice de cópia (idx), sinaliza que está ativa (active=1) e inicia o processo de leitura. Esse comportamento torna o boot determinístico e reiniciável.

O primeiro estágio operacional é o estado de emissão de endereço de leitura (ST_AR). Sua função é apresentar ao barramento o endereço da palavra a ser lida, calculado como SRC_BASE + (idx << 2). A separação desse estado é necessária porque, no protocolo AXI-Lite, o canal de endereço de leitura (AR) é independente do canal de dados (R). O módulo mantém ARVALID ativo até que o slave responda com ARREADY, respeitando a regra de handshake do protocolo. Essa retenção garante robustez contra slaves que introduzem latência.

Após o handshake do endereço, o módulo transita para o estado de recepção de dados (ST_R). Nesse estágio, o sinal RREADY é mantido ativo até que o slave apresente RVALID. A separação entre os estados de endereço e dados é coerente com a arquitetura AXI, onde a aceitação do endereço não implica a entrega imediata do dado. Quando o dado é recebido, ele é armazenado localmente no registrador rd_word, o que desacopla temporalmente a leitura da escrita subsequente.

Concluída a leitura, a FSM passa ao estado de emissão do endereço de escrita (ST_AW). A motivação para um estado exclusivo de endereço de escrita é análoga ao caso da leitura: no AXI-Lite, os canais de endereço (AW) e dados (W) são independentes e possuem handshakes próprios. O endereço de destino é calculado como DEST_BASE + (idx << 2), assegurando alinhamento de 32 bits. O sinal AWVALID permanece ativo até que AWREADY seja recebido.

Em seguida, o módulo entra no estado de envio de dados (ST_W). Nesse ponto, o dado previamente capturado é colocado em WDATA, com WSTRB=4'hF, indicando escrita completa dos quatro bytes. A retenção de WVALID até WREADY assegura que o dado seja

efetivamente aceito pelo slave antes de prosseguir. A separação explícita entre endereço e dado garante conformidade estrita com o protocolo e permite compatibilidade com slaves que aceitem endereço e dado, em ciclos distintos.

Após o envio do dado, a FSM avança para o estado de espera da resposta de escrita (ST_B). Mesmo em AXI-Lite, toda escrita deve ser finalizada com a resposta no canal B. A presença desse estado garante que a transação seja completamente concluída antes que o módulo avance para a próxima palavra. O sinal BREADY é mantido ativo até que BVALID seja recebido, caracterizando o handshake final da operação de escrita. Embora o código não avalie explicitamente o conteúdo da resposta (BRESP), a sincronização garante consistência temporal da transação.

Concluída a escrita, o estado ST_NEXT executa a lógica de controle de fluxo. Ele incrementa o índice idx e decide se o processo deve continuar ou finalizar. Caso o número máximo de palavras tenha sido atingido (idx == WORDS-1), o módulo sinaliza done=1 e transita para o estado final. Caso contrário, retorna ao estado de leitura de endereço, reiniciando o ciclo. Esse estado intermediário existe para separar claramente a conclusão da transação AXI da lógica de decisão do contador, evitando mistura de controle de fluxo com handshakes.

O estado final (ST_FINISH) mantém o módulo em condição estável após a conclusão da cópia. Todos os sinais de controle do barramento permanecem inativos, evitando novas transações. O módulo permanece nesse estado enquanto boot_pin estiver ativo, mantendo done latched. Quando boot_pin retorna a zero, a FSM volta ao estado IDLE, permitindo um novo ciclo de boot no futuro. A existência desse estado impede reinicialização automática indesejada e fornece controle externo claro sobre o ciclo de boot.

Em termos de endereçamento, o módulo acessa duas faixas lineares e contíguas: região de origem iniciando em SRC_BASE e região de destino iniciando em DEST_BASE. Ambas com tamanho total igual a WORDS × 4 bytes. Para o valor padrão de 256 palavras, isso corresponde a 1 KiB transferido da memória de boot para a RAM.

Do ponto de vista arquitetural, a máquina de estados foi implementada de forma sequencial e conservadora, priorizando previsibilidade e conformidade com o protocolo AXI-Lite sobre desempenho máximo. Não há pipeline entre leitura e escrita, nem múltiplas transações pendentes. Essa escolha simplifica a verificação formal e a simulação, além de reduzir o risco de condições de corrida.

Em síntese, cada estado da FSM corresponde diretamente a uma etapa obrigatória do protocolo AXI-Lite: emissão de endereço, transferência de dados e confirmação da operação. A segmentação clara entre esses estados garante robustez, clareza temporal e comportamento determinístico durante o processo de boot por cópia de memória.

6.10 Metodologia de desenvolvimento

Para garantir que o SoC pudesse ter um firmware carregado a partir de uma memória EEPROM externa, uma nova arquitetura contendo o bootloader foi desenvolvida. Ao final do processo de boot, o sistema reinicia para dar seguimento às instruções agora presentes na memória interna.

O processo de desenvolvimento foi realizado em etapas incrementais. Inicialmente foi criada a arquitetura mais simples, sem o bootloader, com o programa sendo carregado diretamente para a memória interna. Essa abordagem permitiu validar a infraestrutura de interconexão e os periféricos de forma isolada. Em seguida, foi desenvolvida uma memória externa acessada por SPI e criado um script para acesso sequencial aos seus endereços, garantindo que o processo de carregamento pudesse ser corretamente exercitado antes da integração ao SoC.

Após a validação do acesso à EEPROM via SPI, foi realizado o desenvolvimento da arquitetura para escrita na memória RAM interna a partir dos dados lidos pela interface SPI, exigindo alterações na arquitetura do top level do SoC para acomodar o novo fluxo de inicialização.

Os testes confirmaram que o bootloader realiza corretamente a operação de leitura dos endereços de memória presentes na EEPROM e os grava na memória interna do SoC. Ao final do processo de boot, o firmware principal é executado normalmente, com o comportamento do sistema mantendo-se idêntico ao observado na arquitetura sem o módulo bootloader — fato verificado pela geração correta do sinal de interrupção do timer ao término da execução.

Conforme esperado, o comportamento do SoC não se alterou após a gravação do firmware via bootloader, sendo que ao final da execução é possível verificar a geração da interrupção no sinal timer_irq, validando o processo de inicialização a partir da memória externa.

6.11 Mapa de memória

O mapeamento de memória adotado na arquitetura com bootloader é apresentado na Tabela 8. A divisão contempla uma janela de memória principal para RAM e ROM, o espaço de

periféricos mapeados via AXI 1×6 e uma janela dedicada de boot que mapeia a leitura para a EEPROM externa via SPI (LARAIA et al., 2026).

Tabela 8 - Mapa de memória do SoC com bootloader

Região / Periférico	MASK	Tamanho	Range (início – fim)	Observação
MEM (região)	0xFFFE_0000	128 KiB	0x0000_0000 – 0x0001_FFFF	Janela total de memória
RAM	0xFFFF_0000	64 KiB	0x0000_0000 – 0x0000_FFFF	Sub-região de MEM
ROM (alias opcional)	0xFFFF_0000	64 KiB	0x0001_0000 – 0x0001_FFFF	Sub-região de MEM
GPIO	0xFFFF_F000	4 KiB	0x4000_0000 – 0x4000_0FFF	AXI 1×6
TIMER	0xFFFF_FF00	256 B	0x4000_1000 – 0x4000_10FF	AXI 1×6
UART	0xFFFF_F000	4 KiB	0x4000_2000 – 0x4000_2FFF	AXI 1×6
SPI (periférico)	0xFFFF_F000	4 KiB	0x4000_3000 – 0x4000_3FFF	AXI 1×6
I2C	0xFFFF_F000	4 KiB	0x4000_4000 – 0x4000_4FFF	AXI 1×6
INTR (IRQ ctrl)	0xFFFF_F000	4 KiB	0x4000_5000 – 0x4000_5FFF	AXI 1×6
BOOT (região)	0xFFFF_0000	64 KiB	0x5000_0000 – 0x5000_FFFF	Janela total de boot
SPI_BOOT (window)	0xFFFF_F000	4 KiB	0x5000_0000 – 0x5000_0FFF	Mapeia leitura para EEPROM

Fonte: Autoria própria.

6.12 Subsistema de periféricos

Desenvolver um subsistema de periféricos modular em Verilog para um SoC com Pico RV32 exige uma estratégia clara de interfaceamento. Onde o objetivo é que cada periférico seja plug-and-play no barramento AXI, permitindo que se adicione ou remova módulos sem reescrever a lógica de controle do processador. O subsistema de periféricos reúne os módulos responsáveis pela interface com o ambiente externo e pela geração de eventos internos utilizados na coordenação funcional do sistema.

Na Tabela 9 tem-se a descrição funcional dos arquivos RTL. Onde cada periférico foi desenvolvido como unidade funcional independente, estruturada em três blocos principais: interface AXI-Lite, banco de registradores e lógica interna de controle (LARAIA et al., 2026).

Essa organização reduz o acoplamento entre protocolo de comunicação e comportamento funcional. Nos módulos de comunicação serial, foram utilizados buffers e FIFOs como blocos auxiliares reutilizáveis, garantindo separação entre plano de controle e fluxo de dados temporizado.

A metodologia adotada privilegiou modularização, reutilização de componentes e isolamento da lógica específica de cada protocolo.

Tabela 9 - Arquivos RTL do subsistema de periféricos

Arquivo (.v)	Módulo Principal	Descrição Funcional
axilgpio.v	axilgpio	GPIO AXI-Lite com registradores de load/set/clear/toggle e IRQ por mudança de estado.
skidbuffer.v	skidbuffer	Buffer de skid para handshake AXI sob backpressure (GPIO).
timer.v	timer	Timer AXI-Lite com compare, prescaler/postscaler, one-shot/autoreload e IRQ.
timer_defs.v	defines	Constantes de endereço e bitfields do timer.
uart_lite.v	uart_lite	UART Lite AXI-Lite (TX/RX) com registradores de configuração, status e dados.
uart_lite_defs.v	defines	Definições de offsets e bits de controle/status da UART Lite.
spi_master_axil.v	spi_master_axil	Wrapper AXI-Lite para controle do SPI master com registradores e FIFOs.
spi_master.v	spi_master	Núcleo SPI master (clock serial, shift TX/RX e controle de chip select).
axis_fifo.v (SPI)	axis_fifo	FIFO AXI-Stream usada no caminho de dados do SPI.
i2c_master_axil.v	i2c_master_axil	Wrapper AXI-Lite do I2C master com controle por software e filas.
i2c_master.v	i2c_master	Motor I2C master de protocolo (START/STOP, ACK/NACK, leitura/escrita).
i2c_init.v	i2c_init	Sequenciador de inicialização I2C para startup.
i2c_single_reg.v	i2c_single_reg	Bloco de registrador de apoio para controle/estado do subsistema I2C.
axis_fifo.v (I2C)	axis_fifo	FIFO AXI-Stream usada no caminho de dados/comandos do I2C.

Fonte: Autoria própria.

6.12.1 GPIO

O módulo GPIO permite configuração de pinos como entrada ou saída e manipulação individual de bits por meio de operações de load, set, clear e toggle. O estado das entradas é refletido nos registradores correspondentes, enquanto alterações configuráveis de nível podem gerar requisições de interrupção. Um buffer de desacoplamento é empregado para estabilização do handshake sob variações de latência.

6.12.2 UART

A UART Lite realiza comunicação serial assíncrona para transmissão e recepção de dados (WOLF, 2012). A taxa de comunicação é definida por divisor programável de clock, e os registradores de status indicam condições operacionais de transmissão e recepção.

6.12.3 SPI

O subsistema SPI distingue interface de controle e núcleo responsável pela geração do clock serial e deslocamento de dados. O fluxo de dados é intermediado por FIFO dedicada, reduzindo a interferência entre temporização do protocolo e transações do barramento.

6.12.4 I2C

O subsistema I2C integra módulo de controle e núcleo responsável pela execução do protocolo, incluindo geração de START, STOP e reconhecimento, conforme especificação oficial (NXP SEMICONDUCTORS, 2014). O fluxo de dados utiliza FIFO para desacoplamento entre controle e temporização do barramento.

6.12.5 Timer

O temporizador implementa contador incremental com suporte a prescaler, postscaler e comparação programável, permitindo operação em modo único ou periódico. A coincidência entre contador e valor configurado resulta em sinalização de interrupção.

6.13 Controlador de interrupção

O Controlador de Interrupções integra eventos assíncronos ao fluxo de execução do núcleo e atua como elemento de coordenação entre periféricos e processamento.

Na Tabela 10 tem-se o arquivo RTL desenvolvido (LARAIA et al., 2026). Onde o módulo foi estruturado mantendo separação entre interface AXI-Lite, banco de registradores de máscara e pendência e lógica de consolidação das fontes de interrupção. Essa organização isola a arbitragem de eventos da lógica funcional dos periféricos.

Quando uma fonte habilitada é acionada, o controlador gera sinal de interrupção ao núcleo. A identificação da origem ocorre por leitura do banco de registradores, e a condição de pendência é removida por escrita controlada. A metodologia adotada segue o mesmo princípio modular aplicado aos demais blocos do SoC, preservando independência estrutural e clareza organizacional.

Tabela 10 - Arquivo RTL do controlador de interrupção

Arquivo (.v)	Módulo Principal	Descrição Funcional
irq_ctrl.v	irq_ctrl	Módulo principal que gerencia múltiplas requisições de interrupção (IRQs). É o bloco responsável por intermediar os eventos externos (periféricos) e a CPU, decidindo qual interrupção será atendida e quando.
irq_ctrl_defs.v	—	Arquivo de definições e constantes utilizadas pelo controlador de interrupções.

Fonte: Autoria própria.

6.14 Verificação funcional do núcleo RISC-V

A presente seção descreve a estratégia de verificação funcional adotada para o núcleo processador RISC-V integrado ao SoC destinado a aplicações em dispositivos de Internet das Coisas (IoT).

A verificação funcional constitui etapa crítica no ciclo de desenvolvimento de circuitos integrados, tendo por objetivo assegurar que a implementação em RTL (Register-Transfer Level) está em plena conformidade com a especificação arquitetural do processador. O trabalho foi estruturado em duas frentes de desenvolvimento independentes e paralelas: a verificação funcional do núcleo RISC-V e a verificação funcional dos periféricos.

A presente documentação concentra-se na primeira frente, abordando os blocos internos do processador, a estratégia de teste adotada, o pseudocódigo do firmware de verificação e o cronograma de execução previsto.

6.14.1 Estratégia de verificação

A abordagem adotada toma como base o desenvolvimento de um firmware de verificação modular, implementado em linguagem C ou Assembly RISC-V, cuja execução em simulação RTL permite a validação sistemática de cada bloco funcional do processador.

A escolha por uma estratégia baseada em firmware permite a reutilização dos casos de teste em diferentes ambientes de simulação e constitui base sólida para a posterior integração com o ambiente UVM desenvolvido nos capítulos seguintes.

As etapas metodológicas comuns a todas as frentes de desenvolvimento compreendem: definição da estratégia de teste; descrição do firmware em pseudocódigo; implementação em linguagem C ou Assembly RISC-V; execução em simulação RTL; e análise de cobertura e validação dos resultados. A execução em FPGA foi prevista como etapa opcional.

A abordagem modular proporciona rastreabilidade clara entre blocos funcionais e casos de teste, viabiliza a validação incremental do núcleo antes da integração com os periféricos e reduz a complexidade durante a fase de depuração.

6.14.2 Escopo da verificação do núcleo

Os blocos internos do processador RISC-V contemplados pelo plano de verificação funcional são: estado de reset e inicialização do sistema; banco de registradores; Unidade Lógica e Aritmética (ULA); unidade de deslocamento (Shift); acesso à memória por meio de instruções de Load e Store; desvios condicionais (Branch); saltos incondicionais (Jump); tratamento de exceções; e tratamento de interrupções, quando aplicável à implementação.

6.14.3 Firmware de verificação

Nesta seção, são abordados pseudocódigo do programa principal e os procedimentos de teste dos blocos verificados. O Firmware de Verificação (muitas vezes chamado de Bare-metal Verification) é um componente híbrido essencial em projetos de chips complexos (SoCs).

6.14.3.1 Estrutura geral

A Figura 3 mostra o pseudocódigo do programa principal, o qual coordena a execução de todos os procedimentos de teste em sequência lógica.

O objetivo do programa principal não é apenas executar uma tarefa, mas sim validar se a infraestrutura de barramento, a decodificação de endereços e o timing dos periféricos estão íntegros.

Figura 3 – Pseudocódigo do programa principal

```
PROCEDIMENTO principal()
    CHAMAR inicializar_ambiente()
    CHAMAR testar_reset()
    CHAMAR testar_banco_registradores()
    CHAMAR testar_ula()
    CHAMAR testar_unidade_shift()
    CHAMAR testar_load_store()
    CHAMAR testar_desvios()
    CHAMAR testar_saltos()
    CHAMAR testar_excecoes()
    CHAMAR testar_interrupcoes()
    CHAMAR reportar_sucesso()
FIM PROCEDIMENTO
```

Fonte: Autoria própria.

6.14.3.2 Procedimentos de teste

A Tabela 11 resume os procedimentos, os blocos verificados e principais asserções. Onde cada procedimento verifica um bloco funcional específico por meio de asserções. A falha de qualquer asserção interrompe a execução e sinaliza o erro correspondente.

Tabela 11 – Procedimentos de teste e blocos funcionais verificados

Procedimento	Bloco Verificado	Principais Asserções
testar_reset()	Reset e estado inicial	PC == vetor de reset; registradores x1–x31 == 0
testar_banco_registradores()	Banco de registradores	Padrões 0xAAAAAAAA e 0x55555555; x0 sempre zero
testar_ula()	ULA	ADD, SUB, AND, OR, XOR, SLT com valores conhecidos
testar_unidade_shift()	Unidade de deslocamento	SLL, SRL e SRA com valores e deslocamentos definidos
testar_load_store()	Acesso à memória	Store/Load de word, halfword e byte
testar_desvios()	Desvios condicionais	BEQ, BNE e BLT tomados corretamente
testar_saltos()	Saltos incondicionais	JAL e JALR direcionam o PC corretamente
testar_excecoes()	Tratamento de exceções	Acesso desalinhado gera MISALIGNED_LOAD
testar_interrupcoes()	Tratamento de interrupções	Interrupção de timer aciona o handler

Fonte: Autoria própria.

6.14.4 Resultados esperados

Ao final da verificação deseja-se alcançar os seguintes resultados: núcleo RISC-V funcionalmente validado em todos os blocos do escopo; falhas identificadas e corrigidas; firmware reutilizável para regressões futuras; e base técnica consolidada para integração com os periféricos e com o ambiente UVM.

Para contextos profissionais, recomenda-se complementar a abordagem com regressão contínua, comparação contra simuladores de conjunto de instruções (ISS) como Spike ou QEMU, e métricas de cobertura funcional.

6.15 Estrutura do ambiente UVM e sequencer

Nesta seção, é abordado o desenvolvimento dos componentes estruturais do ambiente de verificação UVM (Universal Verification Methodology) aplicado ao periférico UART do SoC PicoRV32.

Também, são tratadas a definição da arquitetura geral do ambiente e a implementação do componente Sequencer, responsável pelo controle e ordenação dos estímulos gerados durante a simulação.

O periférico UART verificado é implementado com interface AXI4-Lite de 32 bits de dados e 4 bits de endereço, expondo quatro registradores mapeados em memória: registrador de transmissão TX_DATA (endereço 0x0), registrador de recepção RX_DATA (endereço 0x4), registrador de status STATUS (endereço 0x8) e registrador de controle CTRL (endereço 0xC). O registrador CTRL armazena em seus 16 bits inferiores o divisor de baud rate, utilizado internamente pelo gerador de temporização da comunicação serial.

6.15.1 Arquitetura do ambiente UVM

A arquitetura do ambiente de verificação com base em UVM organiza o processo de verificação funcional de forma estruturada e modular.

A seguir, será dada uma visão geral do ambiente de desenvolvimento e fluxo de verificação funcional, interface de verificação e objeto de transação.

6.15.1.1 Visão geral e fluxo de verificação funcional

A Tabela 12 mostra os componentes UVM e responsabilidades. A metodologia UVM define uma hierarquia padronizada de componentes que compõem o testbench de verificação. O ambiente desenvolvido adota essa hierarquia.

Tabela 12 – Componentes do ambiente UVM

Componente UVM	Responsabilidade
uvm_env	Contêiner principal do ambiente de verificação
uvm_agent	Agrupa interface ativa com o DUT (Driver, Monitor, Sequencer)
uvm_sequencer	Controla ordenação e entrega de transações ao Driver
uvm_sequence	Define cenários de estímulo para o Sequencer
uvm_transaction	Representa uma transação AXI4-Lite individual
uvm_driver	Converte transações em estímulos na interface AXI4-Lite
uvm_monitor	Observar e capturar transações na interface do DUT
uvm_scoreboard	Verifica correção funcional comparando estímulos e respostas

Fonte: Autoria própria.

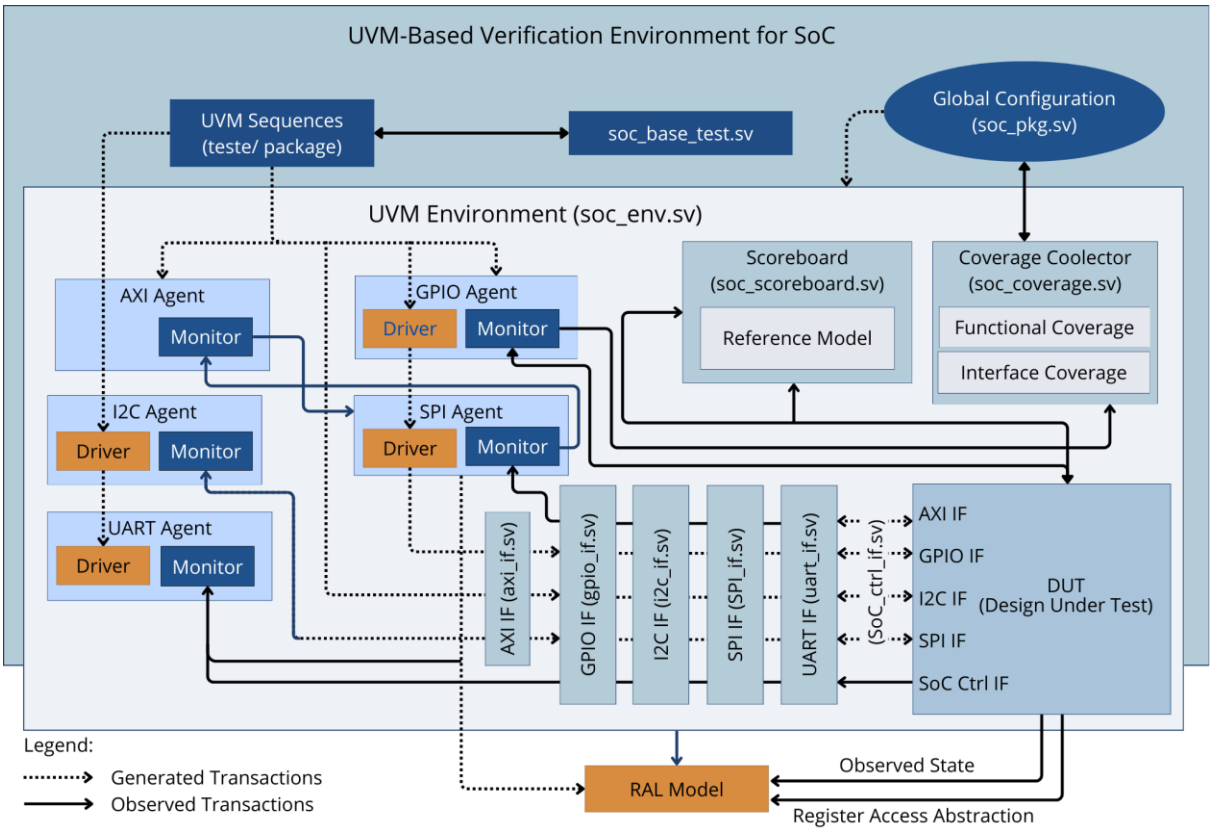
De maneira geral, o fluxo de verificação inicia-se com a geração de estímulos por meio de sequências que produzem objetos de transação, os quais representam operações abstratas do protocolo a ser verificado. Esses objetos são enviados aos componentes do agente, que utilizam uma interface de verificação para aplicar os estímulos ao dispositivo sob teste (Device Under Test – DUT) e observar suas respostas.

A Figura 4 apresenta o fluxo de verificação adotado neste trabalho, desde a geração de estímulos até a análise de coverage e regressão. O fluxo de verificação funcional é estruturado segundo a metodologia Universal Verification Methodology (UVM).

As setas contínuas representam a comunicação por meio de sinais físicos no nível RTL, correspondentes às interfaces do barramento AXI-Lite entre o driver, dispositivo sob teste (DUT) e o monitor.

As setas tracejadas indicam o fluxo de transações em nível de abstração, característico da UVM, no qual objetos de verificação são trocados entre sequências, o modelo de registradores (Register Abstraction Layer – RAL), driver, monitor, scoreboard e os componentes de cobertura.

Figura 4 – Fluxo de verificação multifuncional estruturado do SoC



Fonte: Autoria própria.

6.15.1.2 Interface de verificação

A interface de verificação estabelece a conexão entre o ambiente de verificação e os sinais do hardware descrito em RTL, permitindo a comunicação entre o testbench e o design. Assim, a interface de verificação encapsula todos os sinais AXI4-Lite necessários para a comunicação entre o testbench e o DUT. Os sinais contemplados abrangem os canais de escrita (AW, W e B) e de leitura (AR e R) do protocolo, além dos sinais de clock, reset e das saídas seriais da UART. A interface é instanciada no módulo de topo do testbench e repassada a todos os componentes UVM por meio do mecanismo uvm_config_db.

6.15.1.3 Objeto de transação

Os objetos de transação funcionam como unidades de informação que encapsulam dados e comandos do protocolo, facilitando a geração de cenários de teste, o monitoramento do comportamento do sistema e a análise dos resultados durante o processo de verificação funcional.

Neste trabalho, o objeto de transação estende a classe uvm_sequence_item e representa uma operação atômica sobre o barramento AXI4-Lite. Seus campos compreendem: tipo de operação (leitura ou escrita), endereço de 4 bits, dado de 32 bits e campos de resposta do protocolo (BRESP e RRESP). A classe implementa os métodos utilitários do UVM (do_copy, do_compare, do_print e convert2string), permitindo sua utilização por todos os componentes do ambiente.

6.15.2 Sequencer

Esta seção descreve, de forma concisa, a responsabilidade do componente **sequencer**, as sequências de teste (sequences) implementadas no ambiente UVM e o pseudocódigo do sequencer.

6.15.2.1 Responsabilidade

O **sequencer** é o componente UVM responsável por receber as transações produzidas pelas sequências de teste e entregá-las ao **driver** na ordem e no momento corretos. Ele Atua como árbitro entre múltiplas sequências ativas, implementando políticas de prioridade e garantindo o protocolo de handshake entre **sequencer** e **driver**.

No contexto da verificação da UART AXI4-Lite, o **sequencer** gerencia a entrega de transações de escrita nos canais AW e W e de leitura no canal AR, respeitando as dependências temporais impostas pelo protocolo. O **sequencer** estende a classe uvm_sequencer parametrizada com o tipo do objeto de transação.

6.15.2.2 Pseudocódigo do sequencer e sequências implementadas

A Figura 5 mostra o pseudocódigo que descreve o mecanismo de requisição e entrega de transações ao **driver**. No pseudocódigo, cada sequência (**sequences**) cria objetos de transação, define seus campos (como endereço, dados e operação de leitura ou escrita) e os envia ao **sequencer**. Esse processo é repetido conforme o cenário de teste definido, permitindo verificar operações individuais, sequências de acesso a registradores e condições específicas do protocolo, contribuindo para a cobertura funcional do SoC.

Figura 5 – Pseudocódigo do sequencer

```
PROCEDIMENTO executar_sequence(sequence, sequencer)
    INICIAR sequence no sequencer
    ENQUANTO sequence nao concluida:
        transacao = sequence.gerar_proxima_transacao()
        sequencer.enviar_para_driver(transacao)
        AGUARDAR driver.confirmar_execucao()
        sequence.processar_resposta(transacao)
    FIM ENQUANTO
FIM PROCEDIMENTO

PROCEDIMENTO uart_write_byte_seq(dado)
    transacao.tipo     = ESCRITA
    transacao.endereco = 0x0   // TX_DATA
    transacao.dado     = dado
    ENVIAR transacao
    AGUARDAR STATUS.TX_BUSY == 0
FIM PROCEDIMENTO
```

Fonte: Autoria própria.

Na Tabela 13 tem-se as sequências de teste implementadas da UART em um ambiente UVM. Esses testes visam validar não apenas se os caracteres **saem** pelo pino TX, mas se o protocolo AXI e os buffers internos (FIFOs) suportam as condições de estresse do processador.

As sequências são responsáveis por gerar estímulos que exercitam diferentes funcionalidades do sistema. Portanto, produzem transações e definem os cenários de estímulo a serem executados.

Tabela 13 – Sequências implementadas no ambiente UVM

Sequence	Objetivo	Cenário coberto
uart_reset_seq	Verificar estado pós-reset	Leitura do registrador STATUS após reset ativo
uart_write_byte_seq	Transmitir um byte pela UART	Escrita em TX_DATA e verificação de TX_BUSY
uart_read_byte_seq	Receber um byte pela UART	Leitura de RX_DATA após RX_READY ativo
uart_baud_config_seq	Configurar divisor de baud rate	Escrita em CTRL e verificação do valor programado
uart_loopback_seq	Verificação integrada TX-RX	Transmissão com loopback; comparação TX vs. RX

Fonte: Autoria própria.

6.15.3 Plano de testes

A Tabela 14 apresenta os casos de teste associados à estrutura do ambiente e ao **sequencer.** Os testes foram estruturados segundo a metodologia UVM relacionando casos específicos de teste e critérios de aceitação.

Tabela 14 – Plano de testes do ambiente UVM

ID	Caso de Teste	Critério de Aceitação
TC-01	Inicialização do ambiente UVM sem erros de fase	Fases build, connect e run concluídas sem mensagem fatal
TC-02	Execução da uart_reset_seq	STATUS lido igual a zero após reset ativo
TC-03	Execução da uart_write_byte_seq com byte 0xA5	TX_BUSY ativo durante transmissão; inativo ao término
TC-04	Execução da uart_baud_config_seq com divisor 54	Leitura de CTRL retorna 0x00000036
TC-05	Execução da uart_loopback_seq	Byte transmitido igual ao byte recebido em RX_DATA

Fonte: Autoria própria.

6.16 Driver, monitor e scoreboard do ambiente UVM

Nesta seção aborda-se o desenvolvimento dos componentes ativos do ambiente de verificação UVM que interagem diretamente com a interface do periférico UART: **driver**, **monitor** e **scoreboard**. O **driver** é responsável pela aplicação de estímulos na interface AXI4-Lite do DUT; o **monitor** pela captura passiva das transações observadas; e o **scoreboard** pela verificação automática da correção funcional das respostas produzidas.

O DUT verificado é o periférico UART com interface AXI4-Lite, implementado em Verilog, expondo quatro registradores de 32 bits mapeados em memória e incorporando um gerador de baud rate configurável, um transmissor serial e um receptor serial instanciados e controlados pelo módulo de topo.

6.16.1 Driver

Esta seção descreve, de forma concisa, a responsabilidade do componente **driver**, os testes cobertos pelo driver e o pseudocódigo do driver. O driver é quem efetivamente manipula os sinais elétricos do SoC de acordo com o protocolo (neste caso, AXI-Lite para os periféricos).

6.16.1.1 Responsabilidade

O **driver** atua na camada de abstração entre o mundo das transações (objetos de classe) e o mundo dos sinais (fios e bits). O driver é o componente UVM responsável por receber os objetos de transação fornecidos pelo **sequencer** e convertê-los em estímulos concretos na interface AXI4-Lite do DUT, respeitando rigorosamente o protocolo de **handshake**. Para operações de escrita, conduz os canais AW, W e B. Para operações de leitura, conduz os canais AR e R.

6.16.1.2 Pseudocódigo e testes cobertos pelo driver

O comportamento do **driver**, para uma transação de escrita AXI4-Lite, é descrito pelo pseudocódigo mostrado na Figura 6 e os testes cobertos por ele. O procedimento para transações de leitura segue estrutura análoga, apresentando o endereço no canal AR, aguardando a resposta no canal R e armazenando o dado recebido no objeto de transação para posterior verificação pelo **scoreboard**.

Figura 6 – Pseudocódigo do driver

```
PROCEDIMENTO executar_escrita(34ransação)
    // Canal AW: apresentar 34ransaçã de escrita
    interface.AWADDR  = 34ransação.endereco
    interface.AWVALID = 1
    AGUARDAR interface.AWREADY == 1
    interface.AWVALID = 0
    // Canal W: apresentar dado de escrita
    interface.WDATA  = 34ransação.dado
    interface.WVALID = 1
    AGUARDAR interface.WREADY == 1
    interface.WVALID = 0
    // Canal B: aguardar resposta
    interface.BREADY = 1
    AGUARDAR interface.BVALID == 1
    34ransação.bresp  = interface.BRESP
    interface.BREADY = 0
FIM PROCEDIMENTO
```

Fonte: Autoria própria.

Diferente do **firmware** (que testa o **caminho feliz**), o driver UVM é projetado para testar situações de estresse no protocolo AXI. Nos casos de testes cobertos peo driver, em UVM (Universal Verification Methodology), o driver não os **escolhe** por conta própria. O driver atua como um tradutor, onde sua função é receber transações abstratas (objetos de dados) e

transformá-las em sinais elétricos (bits e temporização) para o design (DUT). No entanto, o driver é o responsável técnico por garantir que certos cenários de protocolo sejam cobertos. A Tabela 15 mostra os tipos de “casos” e situações que o Driver deve cobrir para garantir uma verificação robusta.

Tabela 15 – Operações cobertas pelo driver

ID	Operação	Endereço	Dado / Resposta Esperada
TC-D01	Escrita	0x0 (TX_DATA)	Byte a transmitir; BRESP == OKAY
TC-D02	Escrita	0xC (CTRL)	Divisor de baud rate; BRESP == OKAY
TC-D03	Leitura	0x4 (RX_DATA)	Byte recebido pela UART
TC-D04	Leitura	0x8 (STATUS)	Bits TX_BUSY e RX_READY refletem estado atual
TC-D05	Leitura	0xC (CTRL)	Valor do divisor previamente programado

Fonte: Autoria própria.

6.16.2 Monitor

Esta seção descreve, de forma concisa, a responsabilidade do componente **monitor** e mostra um exemplo de pseudocódigo.

6.16.2.1 Responsabilidade

O Monitor é um componente UVM passivo que observa continuamente os sinais da interface AXI4-Lite sem interferir no comportamento do DUT. Sua função é capturar as transações completadas e publicá-las em uma porta de análise (analysis port) para consumo pelo scoreboard e por outros componentes de cobertura.

A implementação exige atenção especial ao protocolo AXI4-Lite, cujos canais de endereço, dados e resposta são independentes e podem completar em ciclos distintos. O Monitor sincroniza corretamente a captura dos campos para reconstituir a transação completa antes de publicá-la.

6.16.2.2 Pseudocódigo

A Figura 7 mostra um laço de monitoramento para transações de escrita. Onde o monitor precisa observar três canais simultâneos para uma escrita completa: **AW** (Endereço), **W** (Dados) e **B** (Resposta).

Figura 7 – Pseudocódigo do monitor

```
PROCEDIMENTO monitorar_escrita()
    ENQUANTO 36ransação ativa:
        // Capturar canal AW
        AGUARDAR AWVALID == 1 E AWREADY == 1
        36ransação.endereco = interface.AWADDR
        // Capturar canal W
        AGUARDAR WVALID == 1 E WREADY == 1
        36ransação.dado = interface.WDATA
        // Capturar canal B
        AGUARDAR BVALID == 1 E BREADY == 1
        36ransação.bresp = interface.BRESP
        // Publicar 36ransação completa
        porta_analise.escrever(36ransação)
    FIM ENQUANTO
FIM PROCEDIMENTO
```

Fonte: Autoria própria.

6.16.3 Scoreboard

Esta seção descreve, de forma concisa, a responsabilidade, o modelo de referência e o pseudocódigo do componente **scoreboard**.

6.16.3.1 Responsabilidade

O scoreboard é o componente UVM responsável pela verificação automática da correção funcional do DUT. Recebe, por meio de portas de análise, as transações capturadas pelo Monitor e compara os resultados obtidos com os valores esperados calculados pelo modelo funcional de referência do periférico UART.

6.16.3.2 Modelo de referência

O modelo de referência replica o comportamento esperado dos registradores do periférico UART. Para cada escrita no registrador CTRL, o modelo atualiza internamente o divisor de baud rate. Para escritas em TX_DATA, o modelo registra o byte enviado.

Para leituras em RX_DATA, verifica se o byte lido corresponde ao byte transmitido no cenário de loopback. Para o registrador STATUS, verifica a consistência dos bits TX_BUSY e RX_READY com a sequência de operações realizadas.

6.16.3.3 Pseudocódigo

A Figura 8 mostra uma verificação de escrita e compara resultados.

Figura 8 – Pseudocódigo do scoreboard

```
PROCEDIMENTO verificar_transacao(transacao)
    SE transacao.tipo == ESCRITA:
        SE transacao.endereco == 0xC:
            modelo.divisor_baud = transacao.dado[15:0]
        SE transacao.endereco == 0x0:
            modelo.ultimo_byte_tx = transacao.dado[7:0]
        ASSERT(transacao.bresp == OKAY)
    SENAO (LEITURA):
        SE transacao.endereco == 0xC:
            ASSERT(transacao.dado[15:0] == modelo.divisor_baud)
        SE transacao.endereco == 0x4:
            ASSERT(transacao.dado[7:0] == modelo.ultimo_byte_tx)
    FIM SE
    SE assertiva falhou: REPORTAR ERRO com detalhes da transacao
FIM PROCEDIMENTO
```

Fonte: Autoria própria.

6.16.4 Desenvolvimento dos componentes e integração

A Tabela 16 mostra as atividades necessárias para o desenvolvimento dos componentes do ambiente UVM. Essas atividades foram bem definidas e estruturadas sequencialmente.

A integração com os componentes da estrutura UVM e **sequencer** foi prevista e ocorreu na terceira etapa.

Tabela 16 – Etapas de desenvolvimento dos componentes do ambiente UVM

Etapa	Atividade	Componentes
1	Estudo da especificação UART e definição das estruturas de dados	Driver, Monitor, Scoreboard
2	Implementação da lógica principal de cada componente	Driver, Monitor, Scoreboard
3	Integração com o ambiente UVM e primeiros testes de simulação	Todos os componentes
4	Refatoração para AXI4-Lite no contexto do SoC PicoRV32	Driver, Monitor, Scoreboard
5	Integração final e execução de testes de sanidade	Equipe completa

Fonte: Autoria própria.

A abordagem incremental adotada — iniciando pela verificação isolada do periférico UART e progredindo para a validação sistêmica do SoC PicoRV32 com barramento AXI4-Lite — permitiu consolidar o conhecimento sobre a metodologia UVM de forma gradual. Essa estratégia reduz riscos técnicos e facilita a depuração individual antes da integração.

O desenvolvimento em paralelo do **driver**, **monitor** e **scoreboard**, aliado às reuniões internas de alinhamento, garante a compatibilidade entre os componentes e a integração eficiente do ambiente de verificação dentro dos prazos estabelecidos.

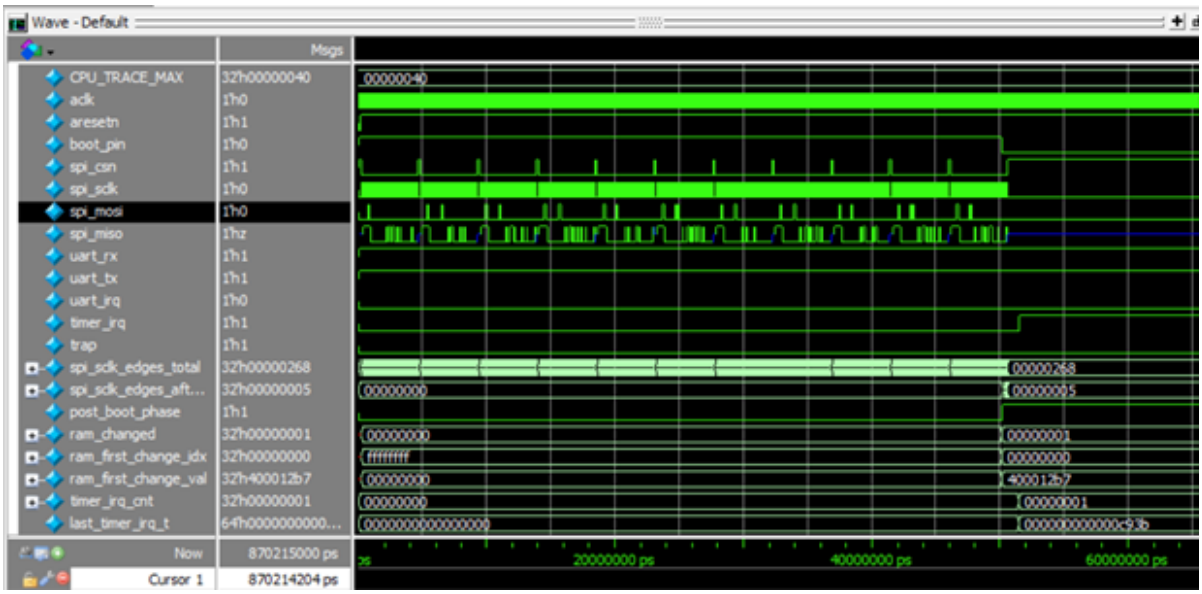
7 RESULTADOS E DISCUSSÕES

Nesta seção, serão apresentados os resultados importantes, durante o desenvolvimento do SoC, para validação desta pesquisa.

7.1 Design

Conforme pode ser observado na Figura 9, o **bootloader** realiza a operação de buscar os endereços de memória presentes na EEPROM e gravá-las na memória interna do SoC. Ao final da execução é possível verificar que após o processo de bootloader o firmware do módulo timer, **timer_irq**, começa a rodar e gera a interrupção como era de se esperar.

Figura 9 – Operações de busca na EEPROM e gravação na memória interna do SoC

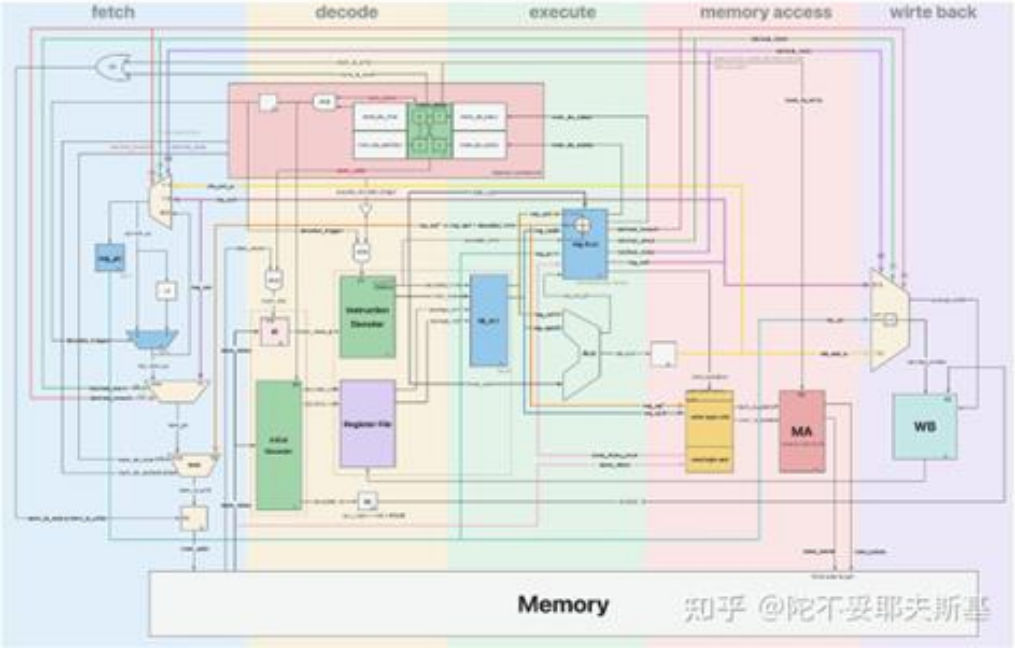


Fonte: Autoria própria.

7.2 Verificação funcional, implementação em FPGA e UVM

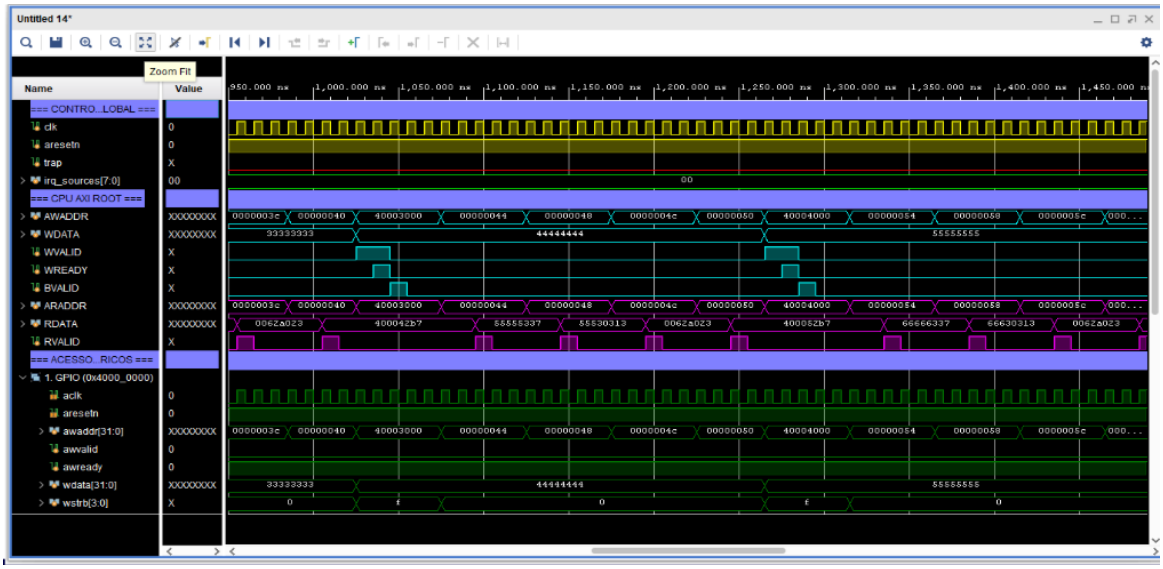
A Figura 10 mostra que o núcleo e os periféricos do SoC foram testados e validados. Contudo, no uso de UVM, identificou-se um erro no firmware da verificação funcional. As Figuras 11, 12, 13, 14 e 15 referem-se aos resultados relacionados à UVM.

Figura 10 – SoC verificado e validado em FPGA



Fonte: Autoria própria.

Figura 11 – Verificação UVM_interconect



Fonte: Autoria própria.

Figura 12 – Relatório da verificação UVM_interconect

```
UVM_INFO C:/AMDDesignTools/2025.2/Vivado/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(19968) @ 2355000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO ../../../../../tb/coverage/soc_coverage.sv(69) @ 2355000: uvm_test_top.env.cov [COV_REPORT] Cobertura Funcional Final: 76.19%
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(96) @ 2355000: uvm_test_top.env.scb [SCB_REPORT] ==================================================
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(97) @ 2355000: uvm_test_top.env.scb [SCB_REPORT]            RELATORIO FINAL DO SCOREBOARD
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(98) @ 2355000: uvm_test_top.env.scb [SCB_REPORT] ==================================================
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(99) @ 2355000: uvm_test_top.env.scb [SCB_REPORT]  Total de Escritas Capturadas: 6
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(100) @ 2355000: uvm_test_top.env.scb [SCB_REPORT]  Total de Leituras Verificadas: 6
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(102) @ 2355000: uvm_test_top.env.scb [SCB_REPORT]  STATUS: PASSED - Nenhuma divergencia encontrada!
UVM_INFO ../../../../../tb/scoreboard/soc_scoreboard.sv(105) @ 2355000: uvm_test_top.env.scb [SCB_REPORT] ==================================================
UVM_INFO C:/AMDDesignTools/2025.2/Vivado/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(13673) @ 2355000: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :   23
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[COV_REPORT]     1
[NO_DPI_TSTNAME]     1
[RNTST]     1
[SCB_REPORT]     7
[SYS_TEST]     3
[TEST]     6
[TEST_DONE]     1
[UVM/COMP/NAMECHECK]     1
[UVM/RELNOTES]     1
[UVMTOP]     1
```

Fonte: Autoria própria.

Figura 13 – Verificação UVM_SoC_Completo

```
covergroup cg_axi_addr_map;
    option.name = "cg_axi_addr_map";

    // Verifica se a CPU acessou todas as faixas de endereços dos periféricos
    cp_address: coverpoint sampled_item.addr {
        // Região de busca de instruções inicial (Reset Vector)
        bins boot_reset  = { [32'h0000_0000 : 32'h0000_0FFF] };
        // Região da RAM
        bins ram_range   = { [32'h0001_0000 : 32'h0001_FFFF] };
        // Periféricos conforme o mapa de endereços
        bins gpio_range  = { [32'h4000_0000 : 32'h4000_0fff] };
        bins timer_range = { [32'h4000_1000 : 32'h4000_1fff] };
        bins uart_range  = { [32'h4000_2000 : 32'h4000_2fff] };
        bins spi_range   = { [32'h4000_3000 : 32'h4000_3fff] };
        bins i2c_range   = { [32'h4000_4000 : 32'h4000_4fff] };
        bins intc_range  = { [32'h4000_5000 : 32'h4000_5fff] };
        bins ram_boot    = { [32'h0000_0000 : 32'h0000_ffff] };
        illegal_bins out_of_bounds = default;
    }

    // Verifica se houve tanto Leitura quanto Escrita em cada periférico
    cp_trans_type: coverpoint sampled_item.trans_type;

    // Cruzamento: Acessamos cada periférico para LER e para ESCREVER?
    cross_addr_type: cross cp_address, cp_trans_type;
endgroup
```

Fonte: Autoria própria.

Figura 14 – Relatório da verificação UVM_SoC_Completo

```
00: reporter [MEM_DATA] EndereÃ§o [0x00000020] : ConteÃºdo = 0x67830313
00: reporter [MEM_DATA] EndereÃ§o [0x00000024] : ConteÃºdo = 0x0062a023
00: reporter [MEM_DATA] EndereÃ§o [0x00000028] : ConteÃºdo = 0x0002a383
00: reporter [MEM_DATA] EndereÃ§o [0x0000002c] : ConteÃºdo = 0x400012b7
00: reporter [MEM_DATA] EndereÃ§o [0x00000030] : ConteÃºdo = 0x3e800313
00: reporter [MEM_DATA] EndereÃ§o [0x00000034] : ConteÃºdo = 0x0062a023
00: reporter [MEM_DATA] EndereÃ§o [0x00000038] : ConteÃºdo = 0x0042a383
00: reporter [MEM_DATA] EndereÃ§o [0x0000003c] : ConteÃºdo = 0x400022b7
00: reporter [MEM_DATA] EndereÃ§o [0x00000040] : ConteÃºdo = 0x04100313
00: reporter [MEM_DATA] EndereÃ§o [0x00000044] : ConteÃºdo = 0x0062a023
00: reporter [MEM_DATA] EndereÃ§o [0x00000048] : ConteÃºdo = 0x0042a383
00: reporter [MEM_DATA] EndereÃ§o [0x0000004c] : ConteÃºdo = 0x400032b7
UN] Sinal de TRAP detectado! Firmware encerrou com sucesso.
RUN] Fim da fase de execÿÿÿÃ£o. Baixando objÿÿÿÃµes.
ackage.sv(19968) @ 665000: reporter [TEST_DONE] 'run' phase is ready to pr
.m_env.m_coverage [COV_FINAL] Total Functional Coverage: 51.85%
_top.m_env.m_scoreboard [SCB_REPORT]



FINAL_STATUS] ************************************
FINAL_STATUS]            TESTE PASSOU
FINAL_STATUS] ************************************
ackage.sv(13673) @ 665000: reporter [UVM/REPORT/SERVER]
```

Fonte: Autoria própria.

Figura 15 – Debug com UVM (na linha amarela deveria ligar o pino 1, mas nada acontece)

Fonte: Autoria própria.

8 CONCLUSÃO

Este trabalho demonstrou com êxito o ciclo completo de desenvolvimento de um System-on-Chip (SoC) voltado para aplicações de IoT. A utilização da arquitetura RISC-V, através do núcleo PicoRV32, aliada ao padrão de comunicação AXI-Lite, resultou em um sistema eficiente e modular. Um dos pilares fundamentais para o sucesso do projeto foi a adoção da metodologia UVM, que proporcionou uma camada de verificação robusta e automatizada, assegurando a integridade do design. A validação em ambiente FPGA confirmou a funcionalidade do SoC, consolidando a viabilidade técnica da solução proposta. Este trabalho estabelece uma base sólida para futuras expansões, como a integração de aceleradores de hardware específicos para **Edge Computing**.

REFERÊNCIAS

ACCELLERA SYSTEMS INITIATIVE. **Universal Verification Methodology (UVM) User's Guide**. Version 1.2. Accellera, 2020.

ARM. **AMBA 4 AXI-Lite Protocol Specification**. Cambridge: ARM Ltd., 2010. Disponível em: https://developer.arm.com/documentation/ihi0022/latest. Acesso em: 18 fev. 2026.

ARM. **AMBA AXI and ACE Protocol Specification**. ARM IHI 0022E. Cambridge: ARM Ltd., 2011. Disponível em: https://developer.arm.com/documentation/ihi0022/latest. Acesso em: 18 fev. 2026.

ATZORI, Luigi; IERA, Antonio; MORABITO, Giacomo. The Internet of Things: A survey. **Computer Networks**, v. 54, n. 15, p. 2787–2805, 2010.

FOSTER, Harry D.; KROPF, Adam C.; LIPPMANN, David. **Assertion-Based Design**. 2. ed. Boston: Kluwer Academic Publishers, 2004.

IEEE. **IEEE Standard for Universal Verification Methodology Language Reference Manual**. IEEE Std 1800.2-2017. New York: IEEE, 2017.

LARAIA, Gabriel Caynã Diniz et al.. **CIDigital-SoC-IoT: CI Digital – projeto SoC para IoT**, 2026. Disponível em: https://github.com/gabriellaraia/CIDigital-SoC-IoT.

MARWEDEL, Peter. **Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems**. 3. ed. Cham: Springer, 2018.

MAXFIELD, Clive. **The Design Warrior's Guide to FPGAs**. Burlington: Newnes, 2009.

NXP SEMICONDUCTORS. **UM10204 I2C-bus specification and user manual**. Rev. 6. Eindhoven: NXP Semiconductors, 2014. Disponível em: https://www.nxp.com/docs/en/user-guide/UM10204.pdf. Acesso em: 18 fev. 2026.

PATTERSON, David A.; HENNESSY, John L. **Computer Organization and Design: The Hardware/Software Interface**. 6. ed. Cambridge: Morgan Kaufmann, 2021.

UFMG. Biblioteca Universitária. **Diretrizes para normalização de trabalhos acadêmicos**. 3. ed. Belo Horizonte: Biblioteca Universitária – Sistema de Bibliotecas da UFMG, 2025.

WATERMAN, Andrew; ASANOVIĆ, Krste. **The RISC-V Instruction Set Manual: Volume I – User-Level ISA**. Version 2.2. RISC-V International, 2019.

WOLF, Clifford. **PicoRV32 – A Size-Optimized RISC-V CPU**. Disponível em: https://github.com/YosysHQ/picorv32. Acesso em: 10 jan. 2026.

WOLF, Wayne. **Computers as Components: Principles of Embedded Computing System Design**. 4. ed. Burlington: Morgan Kaufmann, 2012.